CSCS
Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

Service Accounts in automated workflows

Andreas Fink
ETH Zurich - Swiss National Supercomputing Centre (CSCS)
28th August 2026, Luzern

Proper use cases for service accounts

- SLURM jobs should not run under a specific user in the group
- Run SSH commands from a CI pipeline (e.g. github actions)
- Automate workflows that are triggered outside a login node
- Use FirecREST with a service account
- Use CI/CD provided by CSCS with a service account
- Isolate personal account from automated workflows

Invalid use cases for service accounts

- Avoiding multi-Factor authentication

Requesting a service account

- Service accounts are scoped to a **single project**
- A ticket should be opened on the support portal support.cscs.ch
- Ticket must be opened by PI or a deputy PI
- Describe purpose for service account
- Request template can be found on docs.cscs.ch

Creating a service account

- Self service on portal.cscs.ch after service account has been enabled
- Inside your project: Team --> Add --> Service Account
- Choose a descriptive name
  - Do not prefix it with `service` or `svc`.
  - `svc_` prefix will be added to your selected name
  - A name must be unique across all of CSCS, avoid common names like `cicd`, `automation`
  - An error is raised if you run into a name conflict

Service accounts technical details

- Service account creation will give you an `api key`
- Note down the `api key`, you will not be able to retrieve it again
- The `api key` is valid only for a limited time (180 days) and must be rotated before they expire
- The `api key` is the one and only authentication mechanism
- The `api key` is not the service account user's passwords
  - Web portals that expect a username+password(+OTP) cannot be accessed as the service account user
- SSH keys can be generated, and login via SSH to the cluster is possible
- FirecREST calls can be done
  - CI/CD can use an `api key` to submit jobs with the service account

SSH workflow

- Authentication flow
  1. Request a `JWT` via `api-key`
  2. Sign an SSH key using the JWT for the SSH API service
  3. Login to cluster

SSH workflow details

Exchange API key with JWT

1. Send an HTTP POST request to https://authx-gateway.svc.cscs.ch/api-service-account/api/v1/auth/token
   - Set the header `X-API-Key: $API_KEY` for the request
   - Response is a standard OAuth2 access token response
   - Response is of type JSON and has the field `access_token`, which contains the JWT access token for step 2.
   - ```
     $ API_KEY="<your-api-key>"
     $ AUTH_URL=https://authx-gateway.svc.cscs.ch/api-service-account/api/v1/auth/token
     $ JWT=$(curl --request POST --header "X-API-Key: $API_KEY" $AUTH_URL | jq -r ".access_token")
     ```
     ```

Get a signed SSH certificate

2. Send an HTTP POST request to https://authx-gateway.svc.cscs.ch/api-ssh-service/api/v1/ssh-keys/sign
   - Set the header `Authorization: Bearer $JWT`
   - Send a JSON object of type

```
{
    "publicKey": "$PUBLIC_KEY"
}
```

   - Response is a JSON object which contains the signed certificate in the field `sshKey.publicKey`
   -

```
$ SIGN_URL="https://authx-gateway.svc.cscs.ch/api-ssh-service/api/v1/ssh-keys/sign"
$ PUBLIC_KEY="$(cat /tmp/ssh_key.pub)"
$ JSON_DATA='{"publicKey":"'$PUBLIC_KEY'"}'
$ SIGN_RESPONSE=$(curl --json "$JSON_DATA" --header "Authorization: Bearer $JWT" $SIGN_URL)
$ echo "$SIGN_RESPONSE" | jq -r ".sshKey.publicKey" > /tmp/ssh_key-cert.pub
```

   - Please note that `--json` implies a POST request
   - Please note that the JWT returned in step 1 is valid for 5 minutes

Login to cluster

3. Login to the cluster with the service account username
   - Access is restricted to the same cluster(s) as the user has access to
   - Provide the ssh key
   - Use the service account username
   - Easiest to configure in `$HOME/.ssh/config`

```
Host sa-*
IdentityFile /tmp/ssh_key
User svc_account_name

Host sa-ela
    HostName ela.cscs.ch

Host sa-daint
    HostName daint.cscs.ch
    ProxyJump sa-ela
```

   - Connect to machine

```
$ ssh sa-daint
```

   - Please note that the returned signed certificate is valid only for one minute

Access FirecREST

- FirecREST is an abstraction of the clusters, and offers a rich API to access resources (filesystems, platform status, SLURM)
- Normal FirecREST access is by creating an application on developer.cscs.ch
- Service account cannot login to developer.cscs.ch, because no password is assigned to the user
- Open a support ticket with a request for your service account needing access to the FirecREST API
- Send the FirecREST API call instead of https://api.cscs.ch/hpc/firecrest/v2 to https://f7t-pat.api.svc.cscs.ch/hpcp
- Send the header `X-API-Key: $API_KEY`
- Example

```
$ curl --header "X-API-Key: $API_KEY" https://f7t-pat.api.svc.cscs.ch/hpcp/status/systems
```

Access in CI/CD

- If you use CSCS' CI/CD offering using a service account is straight forward
- Instead of the `Consumer Key` you should use the fixed term `api-key`
- Instead of the `Consumer Secret` you should use your service account's API key

Service Accounts in automated workflows

Live Demo

- Generate service account / rotate API key
- Generate SSH key
- Login to system
- Call FirecREST API
- Look at CI/CD setup